GUÍA MAESTRA DE QUERY METHODS EN SPRING DATA JPA

Teoría completa, sintaxis, reglas de construcción, cuándo usarlos, errores comunes y ejemplos resueltos estilo parcial

Documento de estudio preparado en español, con enfoque práctico para parciales donde cada consulta debe resolverse con un solo método del JPA Repository.

Objetivo	Entender y construir Query Methods correctos sin usar @Query, sin SQL manual y sin repartir la lógica en varios métodos.
Enfoque	Explicación conceptual + patrones de sintaxis + tablas modeladas + soluciones de ejemplo inspiradas en los parciales trabajados.
Idea clave	El nombre del método describe la consulta. Si el nombre coincide exactamente con los atributos de las entidades, Spring Data JPA genera el query automáticamente.

1. Qué es un Query Method

Un Query Method es un método declarado dentro de un repository de Spring Data JPA cuyo nombre describe la consulta que se desea ejecutar. Spring analiza ese nombre, identifica atributos, relaciones, operadores, ordenamientos y límites, y con ello construye el query automáticamente.

En otras palabras: no se escribe SQL, no se escribe JPQL y no se usa @Query. La consulta nace del nombre del método.

- Es ideal cuando el enunciado exige usar únicamente métodos del repository.
- Permite consultas simples y medianamente complejas con relaciones, filtros, orden, top N y validaciones de existencia.
- Exige precisión absoluta en los nombres de los atributos de las entidades.

2. Cuándo usar Query Methods y cuándo no

Los Query Methods funcionan muy bien cuando el problema se puede expresar con filtros sobre atributos o relaciones, operadores comparativos, ordenamiento y un límite de resultados.

Sí conviene usar Query Methods	No conviene usarlos
Filtros por uno o varios atributos	Agregaciones complejas con SUM, AVG, GROUP BY y HAVING
Búsquedas atravesando relaciones con _	Subconsultas o condiciones muy dinámicas
Top/First, OrderBy, Distinct, IgnoreCase	Consultas demasiado largas que ya pierden legibilidad
existsBy, countBy, deleteBy	Lógica donde cambia la consulta según muchos parámetros opcionales

3. Estructura general del nombre del método

La forma mental más útil para construirlos es la siguiente:

```
[TipoRetorno] find | exists | count | delete + First/Top/N + By + condiciones + OrderBy...
```

- El prefijo define la intención: buscar, contar, validar existencia o borrar.
- By inicia la parte del filtro.
- Las condiciones usan nombres exactos de atributos de las entidades.
- Las relaciones se recorren usando guion bajo _.

- El orden se expresa al final con OrderByAtributoAsc o Desc.
- Si el enunciado pide top 5 o top 8, se expresa en el método, no afuera.

4. Prefijos más usados

Prefijo	Ejemplo	Uso
findBy	findByName	Recupera entidades que cumplen una condición.
findFirstBy	findFirstByOrderByDateDesc	Devuelve el primer resultado según el orden indicado.
findFirst5By	findFirst5ByScoreGreaterThan	Devuelve los primeros N resultados.
existsBy	existsByCode	Devuelve boolean para saber si existe al menos un registro.
countBy	countByCountry_Name	Cuenta cuántos registros cumplen el filtro.
deleteBy	deleteByStatus	Elimina por condición.

5. Operadores y palabras clave más frecuentes

Palabra clave	Ejemplo	Significado
And	findByNameAndCountry	Une condiciones con AND.
Or	findByNameOrAlias	Une condiciones con OR.
GreaterThan	findByScoreGreaterThan	Mayor que.
LessThan	findByAgeLessThan	Menor que.
Between	findByDateBetween	Entre dos valores.

IsNull	findByEndDateIsNull	Campo nulo.
IsNotNull	findByPlayersIsNotNull	Campo no nulo; no siempre es necesario.
IgnoreCase	findByNameIgnoreCase	Compara ignorando mayúsculas/minúsculas.
Containing	findByNameContaining	Equivale a LIKE %texto%.
OrderBy...Asc/Desc	findByCountryOrderByScoreDesc	Ordenamiento.
Distinct	findDistinctBy...	Evita duplicados cuando los joins repiten filas.

6. Cómo navegar relaciones con _

El guion bajo _ permite recorrer asociaciones entre entidades. Spring interpreta esa ruta como joins automáticos.

```
findByCountry_Name(String name)
```

Se lee así: partir de la entidad principal, entrar a country y luego comparar su atributo name.

```
findByBus_Route_Type(String type)
```

Se lee así: desde GeoPoint entrar a bus, desde bus entrar a route y filtrar por type.

```
findDistinctFirst5ByPlayerClubs_EndDateIsNullAndPlayerClubs_Player_NameIgnoreCaseOrderByCountry_CodeAsc(String name)
```

Se lee así: desde Club entrar a la colección playerClubs, filtrar solo los registros actuales (endDate null), revisar el jugador asociado por nombre, eliminar repetidos y ordenar por el código del país del club.

7. Regla de oro: el nombre del método usa atributos de la entidad, no columnas físicas

Si la entidad tiene un atributo llamado plate, el método debe usar plate. No se debe escribir el nombre de la columna de base de datos salvo que coincida exactamente con el nombre del atributo Java.

```
private String plate;        -> findByPlate(...)      correcto
private String licensePlate;  -> findByLicensePlate(...) correcto
columna: license_plate       -> findByLicense_Plate(...) incorrecto si el atributo Java no se llama así
```

Por eso, antes de escribir el método, conviene mirar la entidad y anotar la ruta exacta de atributos.

8. Procedimiento práctico para construir un Query Method

1. Identificar la entidad que se quiere devolver. Esa entidad normalmente define en qué repository debe declararse el método.
2. Extraer del enunciado las condiciones, el orden, el top N y si hay que evitar duplicados.
3. Mapear cada condición contra atributos reales de las entidades.
4. Si se debe cruzar una relación, escribir la ruta con _.
5. Elegir el tipo de retorno correcto: List<T>, T, Optional<T>, long o boolean.
6. Agregar OrderBy... y First/Top... si el enunciado lo exige.
7. Revisar si hay joins a colecciones; si sí, considerar Distinct.

9. Errores comunes que bajan nota

Error	Por qué está mal
Olvidar el top N	Si el enunciado dice top 8, el método debe decir findFirst8By...
Ordenar fuera del repository	No cumple la regla de resolver con un solo método del repository.
Usar dos métodos para una sola consulta	El enunciado pide una única consulta resolviendo el problema completo.
Olvidar Distinct	En joins sobre colecciones pueden salir repetidos.
Usar nombres inventados	El método no compila si el atributo no existe.
Poner IsNotNull innecesario	A veces no daña, pero agrega ruido si la condición real ya la garantiza el join.

10. Modelo tipo examen 1 - FIFA World Cup

A continuación se muestra un modelo simplificado de tablas y relaciones, suficiente para entender las soluciones de ejemplo del parcial.

Entidad	Atributos clave	Relaciones	Observación
Country	id, name, code, confederation	1 país tiene muchos players y puede aparecer como home/away en Match	code es útil para ordenar
Player	id, name, fifaScore	Muchos players pertenecen a un country	country es ManyToOne
Club	id, name	Pertenece a un country y se relaciona con PlayerClub	Puede tener varios jugadores en distintos periodos
PlayerClub	id, startDate, endDate	Une Player y Club	endDate null suele representar vínculo actual
Match	id, stadium, date	Tiene homeCountry y awayCountry	Puede filtrarse por estadio y por datos del país local

10.1 Consulta resuelta A - jugadores de una confederación, top 8, orden descendente por fifaScore

Lectura del enunciado:

- Entidad a devolver: Player.
- Filtro: country.confederation = parámetro.
- Orden: fifaScore desc.
- Límite: 8 resultados.

Método correcto:

```
List<Player> findFirst8ByCountry_ConfederationOrderByFifaScoreDesc(String confederation);
```

Por qué funciona:

- findFirst8 indica el límite.
- Country_Confederation recorre la relación desde Player hacia Country.
- OrderByFifaScoreDesc incorpora el orden exigido.

Error típico:

```
List<Player> findByCountry_ConfederationOrderByFifaScoreDesc(String confederation);
```

Ese método filtra y ordena, pero no cumple el top 8.

10.2 Consulta resuelta B - clubes donde actualmente juega un jugador por nombre, top 5, orden por código del país

Lectura del enunciado:

- Entidad a devolver: Club.
- Hay que atravesar la tabla intermedia PlayerClub.
- Actualidad: endDate is null.
- Nombre ignorando mayúsculas.
- Orden por country.code asc.
- Límite 5.

Método correcto:

```
List<Club>
findDistinctFirst5ByPlayerClubs_EndDateIsNullAndPlayerClubs_Player_NameIgnoreCaseOrderByCountry
_CodeAsc(String name);
```

Por qué lleva Distinct:

- Si el join produce varias filas para un mismo club, Distinct evita duplicados en la salida.

Por qué va en ClubRepository:

- Porque el resultado que se quiere devolver es una lista de Club.

Errores típicos:

```
findByDistinctByPlayerClub_Player_NameIgnoreCaseAndPlayerClubEndDateIsNull()
```

Ese nombre está mal armado: usa findByDistinctBy, omite top 5, omite el orden, y además no respeta con claridad la ruta real de atributos.

10.3 Consulta resuelta C - partidos por estadio y país local con jugadores sobre cierto fifaScore

- Entidad a devolver: Match.
- Filtro 1: stadium.
- Filtro 2: homeCountry.name.
- Filtro 3: homeCountry.players.fifaScore > valor.

Método correcto:

```
List<Match>
findDistinctByStadiumAndHomeCountry_NameIgnoreCaseAndHomeCountry_Players_FifaScoreGreaterThan(String stadium, String countryName, Integer fifaScore);
```

Explicación conceptual:

- Desde Match se entra a homeCountry.
- Desde homeCountry se navega a players.
- La base de datos resuelve esto con joins; no significa que Java haga un for manual.
- Distinct evita repetir el mismo partido si varios jugadores del país cumplen la condición.

Versión incompleta que no conviene:

```
findDistinctByStadiumAndHomeCountry_PlayersIsNotNullAndHomeCountry_Players_FifaScoreGreaterThan(...)
```

Aunque se acerca a la idea, le falta filtrar el país específico y usa una condición IsNotNull innecesaria para el enunciado dado.

11. Modelo tipo examen 2 - MetroCali

Modelo simplificado inspirado en los parciales A y B de rutas, buses y geopoints.

Entidad	Atributos clave	Relaciones	Observación
Route	id, name, type	Una ruta tiene muchos buses	name y type aparecen en los enunciados
Bus	id, licensePlate	Muchos buses pertenecen a una route	licensePlate sirve para buscar un bus
GeoPoint	id, latitude, longitude, timestamp	Muchos geopoints pertenecen a un bus	timestamp define recorrido y posición más reciente

11.1 Consulta resuelta A - buses que atienden una ruta por nombre

```
List<Bus> findByRoute_NameIgnoreCase(String routeName);
```

Se devuelve Bus, por eso va en BusRepository. La ruta se recorre con Route_Name.

11.2 Consulta resuelta B - recorrido de un bus por placa, ordenado por tiempo

```
List<GeoPoint> findByBus_LicensePlateOrderByTimestampAsc(String plate);
```

Se devuelve GeoPoint y el orden cronológico se expresa con OrderByTimestampAsc.

11.3 Consulta resuelta C - última ubicación de un bus por placa

```
GeoPoint findFirstByBus_LicensePlateOrderByTimestampDesc(String plate);
```

El primer geopoint al ordenar descendente por timestamp representa la ubicación más reciente.

11.4 Variante del parcial por tipo de ruta

```
List<Bus> findByRoute_TypeIgnoreCase(String type);

List<GeoPoint> findByBus_Route_NameIgnoreCase(String routeName);

GeoPoint findFirstByBus_LicensePlateOrderByTimestampDesc(String plate);
```

La lógica es la misma: identificar la entidad de salida, recorrer relaciones y añadir orden o límite donde haga falta.

12. Modelo tipo examen 3 - Lenguas en peligro

Modelo simplificado inspirado en el parcial de lenguas, hablantes y registros sonoros.

Entidad	Atributos clave	Relaciones	Observación
Language	id, name, classification	Una lengua tiene muchos speakers	classification sirve para filtrar tipo de riesgo
Speaker	id, code, name	Muchos speakers pertenecen a una language	code identifica al hablante
SoundRecord	id, fileName, timestamp	Muchos registros pertenecen a un speaker	timestamp permite hallar el registro más reciente

12.1 Consulta resuelta A - hablantes de lenguas por clasificación

```
List<Speaker> findByLanguage_ClassificationIgnoreCase(String classification);
```

Se devuelve Speaker. El filtro está sobre un atributo de Language.

12.2 Consulta resuelta B - registros sonoros de una lengua específica

```
List<SoundRecord> findBySpeaker_Language_NameIgnoreCase(String languageName);
```

Desde SoundRecord se entra a speaker y luego a language para comparar el nombre de la lengua.

12.3 Consulta resuelta C - registro sonoro más reciente de un hablante por código

```
SoundRecord findFirstBySpeaker_CodeOrderByTimestampDesc(String code);
```

findFirst + OrderByTimestampDesc es el patrón clásico para recuperar el último registro temporal.

13. Tabla de patrones tipo examen

Esta tabla resume patrones frecuentes. Sirve como plantilla mental para construir soluciones rápidas durante el parcial.

Enunciado típico	Patrón del método	Retorno usual	Notas
Entidades por atributo exacto	findByAtributo(...)	List<T>	Si devuelve varias filas.
Entidades por atributo ignorando mayúsculas	findByAtributoIgnoreCase(...)	List<T>	Útil para nombres.
Entidad más reciente por fecha	findFirstBy...OrderByFechaDesc(...)	T u Optional<T>	Muy frecuente.
Top N ordenado	findFirst5By...OrderByScoreDesc(...)	List<T>	Cambiar el 5 por el N pedido.
Validar existencia	existsByAtributo(...)	boolean	No devuelve entidades.
Contar por relación	countByRelacion_Atributo(...)	long	Para métricas simples.
Cruzar entidad hija a padre	findByPadre_Atributo(...)	List<T>	Usa _.
Cruzar con tabla intermedia	findDistinctByIntermedia_Entidad_Atributo(...)	List<T>	Considerar Distinct.
Filtrar vigencia actual	findByEndDateIsNull(...)	List<T>	Muy usado con historial.
Rango de fechas	findByTimestampBetween(...)	List<T>	Recibe dos parámetros.

14. Cómo decidir en qué repository declarar el método

- Si el resultado es List<Player>, normalmente debe ir en PlayerRepository.
- Si el resultado es List<Club>, normalmente debe ir en ClubRepository.
- Si el resultado es un GeoPoint, el método debe vivir en GeoPointRepository.
- La entidad de salida manda más que la entidad desde la cual comienza el recorrido mental del enunciado.

Esta decisión evita métodos forzados o extraños. El repository debe representar a la entidad que retorna.

15. Sobre List<T>, T, Optional<T>, boolean y long

Retorno	Cuándo usarlo	Ejemplo
List<T>	Cuando puede haber varios resultados.	List<Bus> findByRoute_NameIgnoreCase(...)
T	Cuando esperas uno solo y asumes existencia.	GeoPoint findFirstByBus_LicensePlateOrderByTimestampDesc(...)
Optional<T>	Cuando puede no existir ningún resultado.	Optional<GeoPoint> findFirstBy...
boolean	Para existencia.	existsByInterviewerId(...)
long	Para contar.	countBySurvey_Id(...)

16. Cómo se verían estos métodos dentro del repository

```
public interface PlayerRepository extends PagingAndSortingRepository<Player, Integer> {

    List<Player> findFirst8ByCountry_ConfederationOrderByFifaScoreDesc(String confederation);
}
public interface ClubRepository extends PagingAndSortingRepository<Club, Integer> {

    List<Club>
findDistinctFirst5ByPlayerClubs_EndDateIsNullAndPlayerClubs_Player_NameIgnoreCaseOrderByCountry
```

```
_CodeAsc(String name);
}

public interface MatchRepository extends PagingAndSortingRepository<Match, Integer> {

    List<Match>
findDistinctByStadiumAndHomeCountry_NameIgnoreCaseAndHomeCountry_Players_FifaScoreGreaterThan(
            String stadium,
            String countryName,
            Integer fifaScore
    );
}
```

17. Cómo probarlos rápidamente con endpoints

Si el parcial pide exponerlos por RestController, los endpoints deben invocar directamente el método correspondiente del repository, sin agregar lógica extra que rompa la regla del ejercicio.

```
@RestController
@RequestMapping("/partial2jap")
public class Partial2JpaController {

    private final PlayerRepository playerRepository;

    public Partial2JpaController(PlayerRepository playerRepository) {
        this.playerRepository = playerRepository;
    }

    @GetMapping("/players-by-confederation")
    public List<Player> partial2japPlayersByConfederation(@RequestParam String confederation) {
        return playerRepository.findFirst8ByCountry_ConfederationOrderByFifaScoreDesc(confederation);
    }
}
```

La misma idea se replica para ClubRepository, MatchRepository, BusRepository, GeoPointRepository, SpeakerRepository y SoundRecordRepository.

18. Checklist final antes de entregar

- El método resuelve toda la consulta en una sola firma.
- No hay @Query.
- No hay SQL manual.
- No se ordena ni filtra en el service ni en el controller.
- El top N, si existe, está incorporado en el método.
- Los nombres de atributos coinciden exactamente con las entidades.

- Si el join cruza una colección, revisé si necesito Distinct.
- El tipo de retorno coincide con lo que pide el enunciado.

19. Observación importante para adaptar estos ejemplos a tu código real

Los ejemplos de este documento están modelados de forma fiel al estilo de los parciales trabajados, pero siempre debes ajustar el nombre del método a los atributos reales de tus entidades. Si tu clase usa plate en lugar de licensePlate, o routeName en lugar de name, el método también debe cambiar.

Lo que no cambia es la lógica de construcción: entidad de salida, relaciones con _, operadores correctos, orden, límite y Distinct cuando haga falta.

20. Resumen ejecutivo para memorizar rápido

```
1) Identifica qué entidad debes devolver.
2) Pon el método en el repository de esa entidad.
3) Escribe By + filtros usando atributos reales.
4) Recorre relaciones con _.
5) Agrega IgnoreCase, IsNull, GreaterThan, etc. si el enunciado lo pide.
6) Agrega OrderBy al final.
7) Agrega First/Top/N si hay límite.
8) Usa Distinct si el join puede repetir filas.
```

Con esta secuencia puedes traducir la mayoría de los enunciados tipo parcial a un Query Method correcto y defendible.

Fin del documento.